Technical Report: MacroTracker API

Github(Code Repository, API Documentation, Slides): Eursolarmark/WEB-CW1

1. Introduction

This project is a backend API called **MacroTracker API**, developed for a nutrition logging application. The main purpose of the system is to support a simple but realistic user flow: users register and log in, search for food items, record meals by intake weight, and then view summaries and trend-based analytics. In the final version, I intentionally kept the API focused on core functionality instead of trying to implement too many loosely related features. The final endpoint groups cover authentication, food listing and filtering, favourite foods, meal log CRUD operations, daily summaries, and analytics. This design reflects my personal decision to prioritise clarity, consistency, and maintainability over feature quantity.

A key lesson during development was that a good API is not defined by how many endpoints it has, but by whether the endpoints form a coherent product. Earlier versions of the system included additional features such as a dedicated search endpoint, target profile routes, recipe-related ideas, and partial update support. However, after testing and reviewing the user flow, I removed several of these because they overlapped with existing functionality, increased documentation burden, or made the system harder to explain and maintain. This refinement process became one of the most important architectural decisions in the whole project.

2. Technology Stack and Justification

I chose **Python with Django REST Framework (DRF)** as the main framework for this project. My main reason was that DRF provides a strong structure for building APIs while keeping code organised and readable. Serializers make validation explicit and testable, generic views reduce repetitive CRUD boilerplate, and the built-in permission and throttling mechanisms make it easier to enforce endpoint-level rules. For this coursework, I wanted a stack that would help me implement reliable backend behaviour quickly without losing control over validation logic and response design. DRF suited that goal well because it supports rapid development while still encouraging clean architecture.

For authentication, I selected **JWT with SimpleJWT**. This choice allowed the API to remain stateless and suitable for modern client-server interaction. I configured access tokens with a short lifetime and refresh tokens with a longer lifetime, and I enabled refresh rotation with blacklisting. This was a deliberate design choice: it balances usability and security better than a simplistic long-lived token solution. I also improved the login experience by allowing authentication through either username or email while preserving the standard token request structure. This made the API more practical from a user perspective without introducing major complexity.

For the database, I used **SQLite**. I chose it not because it is the most powerful production database, but because it was the most appropriate for this coursework stage. SQLite keeps setup simple, reduces environmental friction, and makes demos and tests easier to reproduce. In the same spirit, I used **drf-spectacular** to support API schema generation and documentation. This tooling combination reflects a practical engineering decision: for a coursework project, reliability and reproducibility in a local environment are more important than premature production-level infrastructure.

3. Key Design and Architectural Decisions

The most significant architectural decision was to keep the API centred around a small number of strong domain models: **FoodItem**, **MealLog**, and **FoodFavorite**. FoodItem stores standardised nutritional data per 100 grams, MealLog records the user's actual intake, and FoodFavorite provides a shortcut layer for repeated food selection. This model structure supports the main user journey without introducing unnecessary abstraction. I wanted the API to be easy to understand both for users and for assessment purposes, so I deliberately avoided a more fragmented model design.

Another important choice was to simplify meal input by accepting only **intake weight in grams**. Earlier, I considered supporting multiple input units, but this quickly introduced avoidable complexity in conversion rules and validation. By standardising the input to grams, the nutrition calculation became deterministic, serializer logic became simpler, and the API contract became easier to explain. I believe this was one of the best simplifications in the project because it improved both implementation quality and user clarity at the same time.

I also consolidated food search into the general /api/foods/ list endpoint with query parameters instead of keeping a separate /api/foods/search/ route. A dedicated search endpoint initially seemed useful, but in practice it duplicated responsibility and increased maintenance overhead. A single listing endpoint with query-based filtering is easier to document, easier to test, and more consistent with REST-style design. In the same spirit, I removed PATCH from the log detail endpoint and kept PUT as the only update method, which reduced ambiguity in client behaviour and simplified the testing matrix. I also removed the separate target profile endpoint because analytics targets could already be handled through query parameters and default values. Together, these decisions reduced the public API surface while keeping the main functional value intact.

A further architectural decision was to compute nutrition values on the server side in MealLog.save(). The API calculates actual calories, protein, carbohydrates, and fat from the food profile and intake grams using deterministic rounding. I chose this approach because it protects data integrity: clients should not be allowed to submit manipulated nutrition totals. It also ensures that analytics endpoints always use consistent computed values, even after a user

updates a food item or a meal log. In my view, this server-side computation is a strong design choice because it places trust and business logic in the backend, where it belongs.

4. Reliability, Testing, and Quality Assurance

Although I simplified the feature scope, I still wanted the API to show engineering quality beyond basic CRUD. Therefore, I implemented endpoint-level throttling, response caching, analytics cache invalidation after writes, idempotency support for selected POST requests, a standardised error envelope, and request tracing through request IDs. These features were included because I wanted the system to demonstrate not only functionality but also operational thinking. Even in a coursework context, a backend should be predictable, traceable, and reasonably resilient.

Testing was a central part of development, especially because the project changed shape several times. I used automated tests as the main regression safety net while refactoring. The test coverage included authentication behaviour, permission boundaries, food filtering, favourites, meal log CRUD operations, nutrition recomputation, analytics outputs, cache invalidation, throttle responses, and standardised error structures. The local full test suite completed successfully with 33 tests passing, which gave me confidence that removing redundant endpoints did not break the remaining core functionality. This testing process strongly influenced my design choices, because it made the cost of unnecessary features very visible.

The repository history also reflects a staged development process rather than a one-time upload. The sequence of commits shows an initial baseline, expanded testing, richer API capabilities, a more polished release, and then final consolidation into a smaller and cleaner product. This version history is important because it demonstrates that the final architecture was not accidental; it was the result of iterative refinement and conscious simplification.

5. Challenges, Limitations, and Future Improvements

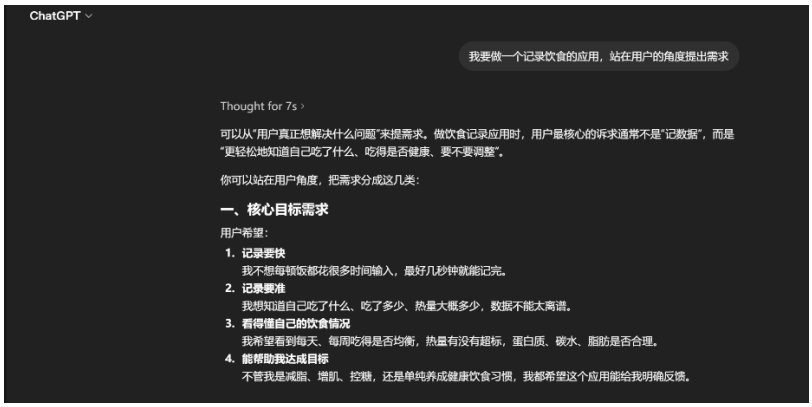
The main challenge in this project was **scope control**. My early instinct was to keep adding features, because adding endpoints can feel like progress. However, I found that each additional route increased the cost of testing, documentation, and explanation. Over time, I learned that coherence matters more than raw feature count. One of the clearest lessons from this project is that good backend design often comes from deciding what not to build. I also encountered the risk of documentation drift: when the API changed quickly, written documentation could become outdated. I addressed this by repeatedly aligning the schema and Markdown documentation with the current implementation before each push.

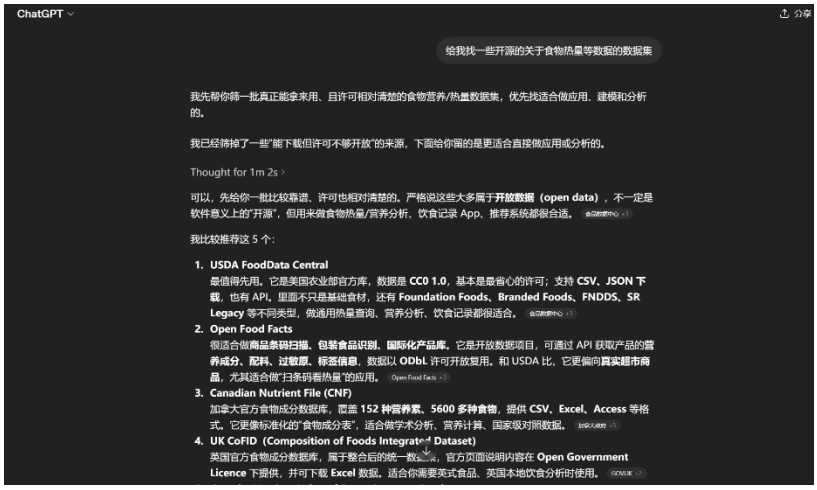
The current system still has some clear limitations. First, it has not yet been publicly deployed, so it remains a locally demonstrated API rather than a production service. Second,

there is no front-end client; the project is intentionally API-first. Third, analytics target handling is still lightweight and mainly query-driven. If I continue developing this project, the next steps would be to deploy it on a managed platform, move to a production-grade database, introduce monitoring and structured logging, and potentially add a recommendation layer based on user history. These improvements would extend the project from a strong coursework backend toward a more complete practical system.

6. Generative AI Declaration

I used **ChatGPT/Codex** as a support tool during the project, mainly for requirement decomposition, architectural trade-off discussion, debugging support, test planning, and documentation polishing. However, I did not use generative AI as an unchecked code generator. Any suggestion that I adopted was manually reviewed, integrated by me, and validated through testing. This means GenAI contributed to my thinking process, but the final implementation decisions and verification remained my own responsibility.





More specifically, GenAI was useful in two meaningful ways. First, it helped me think about the application from the user's perspective, which influenced my decision to prioritise meal logging, summaries, and trends over lower-value endpoints. Second, it helped me explore open nutrition dataset options and think about practical data import and traceability issues. In

both cases, the AI was used as a structured advisory tool rather than a substitute for software engineering judgement. I also applied clear quality controls: I reviewed every major change manually, rejected suggestions that added complexity without enough value, reran the full test suite after major refactors, and kept the final documentation aligned with the implemented routes. This is why I believe my GenAI usage was responsible, transparent, and academically defensible.

7. Conclusion

In conclusion, the MacroTracker API is a deliberately focused nutrition logging backend designed around a clear user flow and a maintainable architecture. My main technical decisions—using Django REST Framework, JWT, SQLite, server-side nutrition computation, and a simplified endpoint structure—were guided by consistency, testability, and clarity. The most important lesson I learned is that software quality often improves when the system becomes smaller and more coherent, not when it becomes larger. Through iterative development, testing, and selective use of generative AI, I was able to produce an API that is modest in scope but strong in structure and engineering reasoning.